Response to Reviewer 1

Enhancing Consent Policy Privacy in Blockchain-Based Healthcare Data Sharing

---

Working draft. Page and line references will be added after the revised manuscript reaches final pagination.

Comments 1: *The security implications of the unbound program length should be more strongly addressed. According to the manuscript, the preimage of the commitment does not include the length of the program ℓ. Thus, a TPS that deviates from the prescribed workflow may consider a stack-consistent prefix of the committed token array instead of the full desired policy. This is not an implementation detail, but a matter of soundness. The proposed framework is oriented toward verifiable consent enforcement and therefore the authors should either bind the program length in the commitment/proof relation in the current implementation or give a much stronger justification for postponing this modification.*

Response 1: We thank the reviewer for identifying this soundness issue. We agree and have bound the program length in the commitment and proof relation in the current implementation.

In the submitted construction, the program length ℓ was used by the interpreter but was not bound to the policy commitment. A deviating TPS that knew the commitment opening could therefore attempt to evaluate a stack-consistent prefix under the commitment to the full padded token array. This can reverse an authorization outcome: for a requester satisfying A, the committed RPN program $[A, \mathrm{NOT}]$ with $\ell = 2$ denies, whereas the prefix $[A]$ with $\ell = 1$ grants.

The revised construction is

$$\begin{aligned} q &= H_{\mathrm{pol}}(P_{\mathrm{enc}}), \\ h &= \mathsf{Com}(P_{\mathrm{enc}}, \ell; r) = \mathsf{Poseidon}(q \parallel \ell \parallel r), \end{aligned}$$

where H_{pol} is the circuit's fixed-length Poseidon aggregation. The same private ℓ is supplied to both the commitment verifier and the RPN interpreter. Consequently, evaluating a different prefix would require opening h to a different (P_{enc}, ℓ) pair, which is excluded by the commitment's binding property under the collision resistance of Poseidon. The length remains private, and this modification adds no public proof input.

We added a dedicated regression suite as regression test that checks the following properties:

(i) the same padded token array and randomizer produce different commitments for different lengths;

(ii) a missing or out-of-range length is rejected;

(iii) the honest $[A, \mathrm{NOT}]$ program denies when $A = 1$;

(iv) its prefix authorizes when separately committed at length one, providing a positive control; and

(v) the truncated length cannot authorize under the commitment to the full program.

All seven regression tests pass.

The isolated circuit change adds 21 non-linear and 67 linear constraints, or 88 constraints in total (approximately 0.12% relative to the submitted 75,097-constraint circuit), and adds no public input. Thus, binding ℓ neither reveals the program length nor changes the policy-independent verification interface. Existing commitments and proving/verification keys from the submitted relation are incompatible with the revised relation and were regenerated.

Changes in the manuscript. The construction is now defined directly in the Design section, its integrity implication is distinguished from the metadata-privacy evaluation in the Discussion, and the former statement that program-length binding remained future work has been removed from the Conclusion.

Location in the revised manuscript: Section "TPS-ZKPI Design," subsection "Commitment Binding and Proof Generation."

> To bind each proof to a provider-defined hidden policy, the provider samples commitment randomness r uniformly from the scalar field and computes
>
> $$q = \mathsf{H}_{\mathrm{pol}}(P_{\mathrm{enc}}),$$
> $$h = \mathsf{Com}(P_{\mathrm{enc}}, \ell; r) = \mathsf{Poseidon}(q \parallel \ell \parallel r).$$
>
> Here, q is the Poseidon-based digest of the fixed-length encoded policy, ℓ is the private program length, r is the commitment randomness, and h is the policy commitment stored on-chain. The semantics of an encoded policy are determined by the pair (P_{enc}, ℓ), since ℓ identifies the active prefix of the padded token array. Accordingly, the circuit supplies the same private value ℓ to both the commitment verifier and the policy interpreter. Under the collision resistance of Poseidon, an accepting proof bound to h cannot substitute a different program length and therefore cannot evaluate an alternative stack-consistent prefix of P_{enc}.

Location in the revised manuscript: Section "Discussion."

> As described in the Design section, the policy commitment covers both the padded encoding P_{enc} and its active length ℓ. Because ℓ also determines how many tokens the interpreter evaluates, an authorizing proof for h cannot be based on a shorter valid prefix. This integrity guarantee is separate from our privacy evaluation, which measures whether public-chain artifacts reveal information about the hidden policy.

Comments 2: *The proof must be cryptographically linked to the authorization context. The current proof statement is not linked to the requester identity, protected resource, or session context. As mentioned in the manuscript, single-use of the nonce does not allow reuse of the same session parameters, but it does not prevent replay of a valid proof captured by an active party using fresh parameters. This is a significant limitation of the authorization guarantees of the current prototype. The authors would ideally add the relevant requester/resource/session values to the proved statement and experimentally evaluate the resulting circuit/public-input and gas cost changes. If this cannot be implemented in the current version, the security claims and deployment implications should be made even more explicit throughout the manuscript, including the abstract and conclusion.*

Response 2: We thank the reviewer for identifying this authorization-context gap. We agree and have implemented the proposed binding in both the circuit and the on-chain access-control path.

In the submitted system, the public statement contained the authorization output, policy commitment, and parameter commitment, but no value derived from the requester, resource, or session. The contract checked nonce reuse only from the transaction parameters. Consequently, a captured proof could be paired with a fresh nonce or another resource while the proof itself remained unchanged.

For the revised construction, let

$$\chi = (id_{\text{req}}, id_{\text{res}}, \nu), \qquad d_\chi = \mathsf{Poseidon}(id_{\text{req}} \parallel id_{\text{res}} \parallel \nu),$$

where ν is the session nonce. The circuit takes the three components of χ as private circuit inputs and enforces the hard equality $d_\chi = \mathsf{Poseidon}(\chi)$, with d_χ exposed as a public input. The ACM receives the same three values as transaction parameters, recomputes the digest on-chain, and requires it to equal the proof's public signal before verification. It also checks the resource–policy association, the public request epoch, and nonce single-use for the tuple $(\mathit{policy}, \mathit{resource}, \nu, id_{\text{req}})$. Thus, within one ACM deployment, changing the requester identifier, resource, or nonce causes rejection.

The three context values are private with respect to the Groth16 relation but are not hidden from the blockchain: they are present in transaction calldata so that the ACM can recompute the digest. We therefore describe id_{req} as a requester identifier or pseudonym, not as a hidden or universally authenticated real-world identity. The current statement contains five signals:

$$(1, h, pp, d_\chi, \tau),$$

namely one output and four public inputs; τ is a separate request-epoch input added for temporal policy checks.

We measured the context-binding change in isolation against the length-bound circuit immediately preceding it. It adds 605 R1CS constraints, increasing the total from 75,185 to 75,790 (0.8%), and adds one public input.

The revised implementation is covered at both layers. At the circuit layer, the test rejects a context digest computed for another nonce. At the contract layer, the tests confirm agreement between the circuit-side and EVM Poseidon implementations and reject substitutions of the requester, resource, and nonce. The end-to-end suite reuses one valid proof as a negative control and confirms that a fresh nonce or another resource governed by the same policy fails with a context-binding mismatch. All targeted tests pass.

We have kept the claim precise: the change closes the reviewer's fresh-parameter replay within one ACM deployment. The digest does not currently include the transaction submitter or a chain/contract domain, so the revised Discussion identifies exact-context front-running and cross-deployment reuse as separate hardening considerations.

Changes in the manuscript. The Design section now defines the context digest and the five-signal statement, explains the circuit and ACM checks, and removes the former future-work language. The Problem and Threat Model section now distinguishes the issuer-authenticated profile claims from other evidence, separates the pseudonymous requester identifier from real-world identity and issuer governance, and identifies adaptive requester probing and off-chain side channels as outside the present evaluation. The response reports the measured marginal cost, while the Discussion states the scope of the resulting guarantee. The cost profile now reports the implemented circuit and public interface together with the revised Polygon Amoy gas, calldata, and prover measurements.

Location in the revised manuscript: Section "Problem and Threat Model," subsection "System and Trust Model."

> Attribute-origin guarantees depend on the evidence type. For DUO terms evaluated from the claims in an issuer-signed profile credential presented by the requester (hereafter, profile-backed terms), the prototype verifies the EdDSA-Poseidon credential under an issuer key selected by the provider and bound to the policy commitment. The credential binds the policy commitment, requester pseudonym, protected resource, claims, validity interval, and credential identifier; the corresponding policy attributes are derived inside the circuit. Thus, subject to signature unforgeability and Poseidon collision resistance, these values cannot be modified without invalidating the proof. This guarantee does not establish the real-world authority of the issuer or the truthfulness of issuance. Ontology and requester-allowlist predicates likewise establish membership only relative to provider-committed sets. Issuer-key governance, credential revocation, real-world identity verification, and protection against a compromised TPS remain external trust and lifecycle functions.

Location in the revised manuscript: Section "Problem and Threat Model," subsection "Public-Chain Adversary."

> Adaptive requester probing and off-chain side channels are outside the scope of the present evaluation. The proved authorization outcome is bound to a pseudonymous requester identifier, a protected resource, and a session nonce through the context digest defined in Section 5. This binding prevents substitution of the recorded authorization context, but it should not be interpreted as proof of the requester's real-world identity.

Location in the revised manuscript: Section "TPS-ZKPI Design," subsection "Commitment Binding and Proof Generation."

> For an access request, let
>
> $$\chi = (id_{\text{req}}, id_{\text{res}}, \nu), \qquad d_\chi = \mathsf{Poseidon}(id_{\text{req}} \parallel id_{\text{res}} \parallel \nu),$$
>
> where ν is a fresh session nonce. Here id_{req} is a pseudonymous authorization identifier. For policies that activate a holder-bound credential or requester-allowlist predicate, the circuit derives it as $\mathsf{Poseidon}(s_{\text{holder}})$ from a private holder secret s_{holder}; otherwise, it may be a requester-supplied pseudonym. The TPS constructs the private circuit witness
>
> $$w = (P_{\text{enc}}, \ell, a, b, r, \omega, \chi).$$
>
> The context tuple χ consists of private circuit inputs, while its digest d_χ is public. The current public-signal vector is
>
> $$(1, h, pp, d_\chi, \tau),$$
>
> where pp is a fixed-format commitment to the per-predicate parameters and τ is the request epoch used by the temporal checks. Although the three components of χ are not separate Groth16 public inputs, the ACM receives them as transaction parameters in order to recompute d_χ; they are therefore visible in transaction calldata. The TPS generates a proof
>
> $$\pi \leftarrow \mathsf{Prove}(pk, w; (1, h, pp, d_\chi, \tau)).$$
>
> The proof certifies the conjunction that $C(P_{\text{enc}}, \ell, (a, b)) = 1$, $h = \mathsf{Com}(P_{\text{enc}}, \ell; r)$, $d_\chi = \mathsf{Poseidon}(\chi)$, and the selected external-predicate witnesses satisfy their policy-bound verification relations. Before verifying π, the ACM checks that the resource

> is governed by the stated policy, that τ is current, that the nonce has not been used for the same requester–resource context, and that the digest recomputed from the transaction parameters equals the public signal d_χ. Thus, within one ACM deployment, a proof generated for one requester identifier, resource, and nonce cannot be accepted under a different tuple.

Location in the revised manuscript: Section "Discussion," paragraphs on authorization-context binding and its deployment scope.

> The authorization context is likewise part of the proved relation. The circuit constrains the public digest d_χ to the requester identifier, resource, and nonce used in the witness, and the ACM recomputes the same digest from the transaction parameters. Consequently, changing any of these values invalidates the request.
>
> The guarantee is scoped to the bound tuple within one ACM deployment. The digest does not include the transaction submitter or a chain/contract domain, so exact-context front-running and cross-deployment reuse require separate deployment-level controls. Moreover, id_{req} is a pseudonymous authorization identifier and is not, by itself, proof of a real-world identity or of the origin of every private attribute.

Location in the revised manuscript: Section "Privacy Evaluation," Table "Single-point cost profile of the evaluated prototype configuration at $\mathsf{DEPTH} = 64$."

Single-point cost profile of the evaluated prototype configuration at $\mathsf{DEPTH} = 64$. Gas, logs, calldata, and local prover timings are summarized over the 1,773 revised Polygon Amoy executions.

Metric	Value / source
Circuit size	81,740 R1CS constraints; 80,884 wires; $\mathsf{DEPTH} = 64$
Public interface	4 public inputs plus 1 output
Padded on-chain gas	Commit 304,167; access 1,057,283 mean gas
Event logs	Commit: 1 log / 180 bytes; access: 2 logs / 296 bytes
Calldata length	Commit 196 bytes; access 612 bytes (constant across policies)
Local witness time	763.6 ms mean
Local prove time	2,884.2 ms mean

Comments 3: *The problem of attribute-origin requires further investigation. The issuer-signature mechanism is a placeholder for consistency, based on Poseidon, and thus does not provide cryptographic unforgeability as it stands. Thus, the proof proves the consistency of the given witness but does not prove that the private attributes are derived from an authorized issuer. This difference is particularly important in a healthcare authorization system. The authors should clarify which authorization properties are guaranteed by the prototype they implemented, and which are dependent on external trust. Even an analytical assessment of a concrete design for the integration of real credential verification would greatly improve the work.*

Response 3: We thank the reviewer for emphasizing this distinction. We agree that witness consistency alone is insufficient to establish attribute origin, and that a healthcare authorization prototype must separate cryptographic credential verification from trust in the real-world issuer.

During the revision, we clarified and documented the implemented in-circuit EdDSA-Poseidon verification path for a credential carrying fixed DUO-relevant claims. For the private, issuer-signed claim bits $c_0, \ldots, c_6$ for, respectively, research use, health or medical use, methods development, genetic study, ancestry, commercial use, and non-profit use, let d_{DUO} denote the fixed credential schema/domain tag. The circuit reconstructs

$$m_{\text{cred}} = \mathsf{Poseidon}(d_{\text{DUO}} \parallel h \parallel id_{\text{req}} \parallel id_{\text{res}} \parallel c_0 \parallel \cdots \parallel c_6 \parallel t_{\text{from}} \parallel t_{\text{until}} \parallel \gamma)$$

Before proof generation, an issuer computes $\sigma \leftarrow \mathsf{Sign}_{\text{EdDSA-Poseidon}}(s_{\text{iss}}, m_{\text{cred}})$ with its private signing key s_{iss}. The resulting credential is supplied to the TPS as private off-chain evidence for proof generation. At proof generation, the TPS supplies the credential as private witness and the circuit verifies the issuer's EdDSA-Poseidon signature on this message. It also constrains the supplied issuer point to the issuer-key commitment contained in the policy-bound parameters, enforces the credential validity interval and a non-zero credential identifier, and, for a policy that activates this holder-bound credential slot, binds the requester pseudonym to a holder secret. The circuit derives the DUO terms GRU, HMB, NMDS, GSO, NPUNCU, NPOA, NPU, and NCU from these signed claims. Under the unforgeability of EdDSA for correctly generated Baby Jubjub keys and the collision resistance of Poseidon, a TPS cannot modify the signed policy, requester, resource, claims, validity interval, or credential identifier and still produce an accepting proof.

We have also narrowed the claim carefully. The prototype proves signature validity under a policy-selected issuer key; it does not prove that this key belongs to an accredited hospital, ethics board, or other healthcare authority. The provider currently selects the issuer-key commitment, and there is no independently governed issuer registry or credential-revocation check. Ontology predicates prove that a supplied code belongs to a provider-committed set, while requester allowlists prove that a holder pseudonym belongs to a provider-committed list; neither mechanism independently authenticates every associated real-world fact. Furthermore, the current TPS receives the holder secret and credential witness, so holder binding does not protect against a malicious TPS reusing otherwise valid evidence within its signed scope. The Polygon Amoy evaluation uses a deterministic test-only issuer key to exercise the cryptographic path and measure the complete system; it is not evidence of institutional issuance or production key management.

We added an analytical production-integration design to the Discussion. A governed issuer registry would publish schema-qualified, time-bounded issuer keys and require canonical encoding, subgroup validation, and proof of possession at registration. The issuer would sign a domain-separated credential containing a holder-key commitment, claims, authorization scope, validity interval, and credential identifier. The circuit would verify the credential signature and private membership of the issuer key in the active registry. The holder would retain its secret and sign the authorization context for each session, preventing the TPS from creating fresh-session proofs from a captured credential presentation. An epoch-specific sparse-Merkle or accumulator root would additionally support private non-revocation proofs. These roots can be folded into a fixed-format state commitment, preserving the policy-independent public interface. For credentials using Ed25519, ECDSA, or another non-circuit-friendly signature, the alternatives are to pay the higher in-circuit verification cost or have the issuer additionally issue a SNARK-friendly credential; TPS re-signing would not preserve issuer-origin assurance.

The implemented credential path is covered by the DUO profile and tamper suites. The targeted tests accept a correctly signed credential and reject a non-committed issuer, a forged signature under the committed key, credentials outside their validity interval, substitutions of the policy, requester, resource, or signed claims, inconsistent derived attributes, and an invalid holder binding. All 25 targeted tests pass, including full Groth16 proof and on-chain grant/deny cases, and the frozen manifest, verifier ABI, and cryptographic artifacts validate successfully.

Changes in the manuscript. The Problem and Threat Model now separates issuer-signed profile claims from ontology and allowlist evidence and states the remaining issuer-governance, revocation, identity, and TPS trust assumptions. The Design section replaces the obsolete placeholder description with the implemented credential relation and its exact scope. The Discussion distinguishes cryptographic signature validity from real-world issuer authorization and presents a concrete registry, holder-proof, and revocation integration design.

Location in the revised manuscript: Section "Problem and Threat Model," subsection "System and Trust Model."

> Attribute-origin guarantees depend on the evidence type. For DUO terms evaluated from the claims in an issuer-signed profile credential presented by the requester (hereafter, profile-backed terms), the prototype verifies the EdDSA-Poseidon credential under an issuer key selected by the provider and bound to the policy commitment. The credential binds the policy commitment, requester pseudonym, protected resource, claims, validity interval, and credential identifier; the corresponding policy attributes are derived inside the circuit. Thus, subject to signature unforgeability and Poseidon collision resistance, these values cannot be modified without invalidating the proof. This guarantee does not establish the real-world authority of the issuer or the truthfulness of issuance. Ontology and requester-allowlist predicates likewise establish membership only relative to provider-committed sets. Issuer-key governance, credential revocation, real-world identity verification, and protection against a compromised TPS remain external trust and lifecycle functions.

Location in the revised manuscript: Section "TPS-ZKPI Design," subsection "Credential-Origin Binding."

> The circuit reserves one fixed predicate slot for a credential carrying a fixed set of DUO-relevant claims. Let $c_0, \ldots, c_6$ be the private, issuer-signed claim bits for, respectively, research use, health or medical use, methods development, genetic study, ancestry, commercial use, and non-profit use. Let t_{from} and t_{until} define its validity interval, and let γ be its credential identifier. Let d_{DUO} be the fixed credential schema/domain tag for this credential type. From these signed claims, the circuit derives the DUO terms GRU, HMB, NMDS, GSO, NPUNCU, NPOA, NPU, and NCU. For issuer public key $A = (A_x, A_y)$, the circuit reconstructs the domain-separated message
>
> $$m_{\text{cred}} = \mathsf{Poseidon}(d_{\text{DUO}} \parallel h \parallel id_{\text{req}} \parallel id_{\text{res}} \parallel c_0 \parallel \cdots \parallel c_6 \parallel t_{\text{from}} \parallel t_{\text{until}} \parallel \gamma)$$
>
> Before proof generation, an issuer holding the corresponding private signing key s_{iss} issues the credential by computing
>
> $$\sigma \leftarrow \mathsf{Sign}_{\text{EdDSA-Poseidon}}(s_{\text{iss}}, m_{\text{cred}}).$$
>
> The resulting credential is supplied to the TPS as private off-chain evidence for proof generation. At proof generation, the TPS supplies A, σ, and the credential fields as private witness values. The circuit then enforces
>
> $$\mathsf{Verify}_{\text{EdDSA-Poseidon}}(A, \sigma, m_{\text{cred}}) = 1, \qquad \mathsf{Poseidon}(A_x \parallel A_y) = k_{\text{iss}}$$
>
> where k_{iss} is the issuer-key commitment contained in the policy-bound parameter commitment pp. The circuit additionally requires $t_{\text{from}} \le \tau \le t_{\text{until}}$ and $\gamma \neq 0$. Because this credential is holder-bound, a policy that activates this slot also requires $id_{\text{req}} = \mathsf{Poseidon}(s_{\text{holder}})$, binding the credential to the pseudonymous requester identifier in the authorization context. The corresponding DUO attributes are constrained to these claim-derived values.

Location in the revised manuscript: Section "Privacy Evaluation," subsection "Dataset and corpus generation."

> For reproducibility, the evaluation generator uses a deterministic test issuer to create the private profile-credential witnesses used by policies that exercise the credential predicate; this testing key does not model issuer governance or production key management.

Location in the revised manuscript: Section "Discussion," paragraphs on the implemented attribute-origin guarantee and its remaining trust anchor.

> Attribute origin is only partially delegated to the TPS. For the profile-backed DUO terms, the circuit verifies the EdDSA-Poseidon signature described in the Design section and derives eight policy attributes from the seven signed claims. A compromised TPS cannot modify the signed claims, policy commitment, requester pseudonym, resource, validity interval, or credential identifier without invalidating the signature. The implemented relation rejects a non-committed issuer, an invalid signature under the committed key, an expired or premature credential, and substitutions of any signed field. Other evidence has a narrower meaning: ontology proofs establish that a supplied code belongs to a provider-committed set, and requester allowlists establish that a holder pseudonym belongs to a provider-committed list; they do not authenticate every underlying real-world fact. Moreover, because the TPS receives the holder secret and credential witness in the current workflow, holder binding does not protect against a malicious TPS reusing otherwise valid evidence within its signed scope.
>
> The remaining trust anchor is therefore issuer governance. The prototype binds the proof to a policy-selected issuer-key commitment, but it has no independent registry establishing that this key belongs to an accredited healthcare authority, and it does not check credential revocation. The evaluation uses a deterministic test-only issuer key to exercise the complete signature path; it is not evidence of institutional issuance or production key management.

Location in the revised manuscript: Section "Discussion," paragraphs presenting the production credential-integration design.

> A concrete deployment can close this gap without changing the Boolean interpreter. First, a governed issuer registry would publish schema-qualified issuer keys and their validity epochs; registration would validate a canonical, subgroup-safe public key and require proof of possession. Second, an issuer would sign a domain-separated credential containing a holder-key commitment, claim vector, authorization scope, validity interval, and unique credential identifier. Third, the circuit would verify both the credential signature and a Merkle proof that the issuer key is active in the registry, while continuing to derive policy attributes from the signed claims. The holder would retain its secret key and authorize each request by signing the context digest and request epoch; verifying this second signature inside the circuit would prevent a TPS that observes one credential presentation from generating proofs for fresh sessions. Finally, an epoch-specific sparse-Merkle or accumulator root would support a private non-revocation witness for the credential identifier. The registry and revocation roots can occupy fixed-format public or committed parameter slots, preserving the Boolean interpreter and the shape of the policy-independent public interface; integrating these checks would nevertheless require a revised circuit and new proving and verification keys. If an existing healthcare credential uses a non-circuit-friendly signature such as Ed25519 or ECDSA, integration requires either verifying that signature in-circuit at higher constraint cost or having the issuer additionally issue a SNARK-friendly credential; re-signing by the TPS would not preserve issuer-origin assurance.

Comments 4: *The trusted proof service is still an assumption of trust. The privacy contribution should be viewed in the context that the TPS has access to confidentiality-sensitive information and builds the witness. The implementation also co-locates the TPS with the provider, concentrating this trust. Thus, a compromised or colluding TPS may replace attributes or refuse to compute a proof. This problem is acknowledged in the discussion, but the practical implications require a more comprehensive discussion. The authors should clarify the difference between the cryptographic guarantees provided by TPS-ZKPI and the guarantees based on honest TPS behavior. A small threat/guarantee matrix would help clarify things.*

Response 4: We agree, and we have made the distinction explicit in two places.

First, the trust-model subsection now separates the two kinds of guarantee in the text. Authorization correctness is enforced by the commitment and the on-chain proof verification: a proof that does not satisfy the committed relation is rejected regardless of how the TPS is instantiated, and, following the revisions for Comments 1–3, the relation now binds the program length, the authorization context, and the issuer-signed profile claims, so a compromised TPS cannot shorten the program, replay a proof under fresh parameters, or alter signed claims. Policy confidentiality, in contrast, is not cryptographically enforced against the TPS: the TPS holds the plaintext policy material and the requester evidence, and a compromised or colluding TPS can expose them, substitute unsigned evidence, or refuse to prove. The provider-operated TPS used in the evaluation concentrates this trust at the provider.

Second, we added the requested matrix (the threat/guarantee table in the revised Problem and Threat Model section). Its rows distinguish the passive public-chain observer, untrusted requester, workflow-following TPS, compromised or colluding TPS, forged credentials, and unauthorized or revoked issuers; its columns cover policy confidentiality, authorization correctness, liveness, and decision auditability. Each cell states whether the guarantee is cryptographic (C), rests on the TPS following its workflow (T), is an external or lifecycle function (E), is not provided (–), or is not applicable (n/a). The matrix makes the practical implications visible at a glance: against a compromised TPS, the prototype retains authorization correctness for the committed relation and signed claims but loses confidentiality and liveness; submitted decisions remain auditable, whereas a refusal to prove is not attributable. A note below the table states that the evaluated provider-operated deployment concentrates every TPS-dependent assumption at the provider.

Changes in the manuscript. The revised trust model states the cryptographic and TPS-dependent guarantees separately, the new matrix makes the distinction explicit for every threat, and the Discussion identifies the practical consequences and reduced-trust directions.

Location in the revised manuscript: Section “Problem and Threat Model,” subsection “System and Trust Model,” paragraphs on the provider-operated TPS.

> For implementation and evaluation simplicity, the current evaluation co-locates the TPS with the provider (a provider-operated TPS). This co-location concentrates trust at the provider side: a single party now holds both the policy and the requester evidence, so a compromised provider-operated TPS could substitute inputs whose origin is not independently authenticated or withhold proof generation from legitimate requesters. Two guarantees must be distinguished here. Authorization correctness is primarily enforced by the commitment and on-chain proof verification: a proof that does not satisfy the committed policy relation is rejected, largely independently of how the TPS is instantiated. Policy confidentiality, in contrast, is not cryptographically enforced against the TPS itself; it rests entirely on the trust assumption stated above. Our evaluation therefore assumes a TPS that follows the prescribed workflow and isolates the remaining question: what a public-chain observer

can infer from the resulting public artifacts. Weakening the TPS trust assumption itself is orthogonal and is the role of the reduced-trust instantiations above. Table 3 summarizes, for each actor or failure mode, which of the four guarantees—policy confidentiality, authorization correctness, liveness, and decision auditability—is enforced cryptographically, which holds only under the TPS trust assumption, and which the prototype leaves to external functions or does not provide.

Location in the revised manuscript: Section "Problem and Threat Model," new Table 3, "Guarantees under each actor or failure mode."

Table 3. Guarantees under each actor or failure mode.

Actor or failure mode	Policy confidentiality	Authorization correctness	Liveness	Decision auditability
Passive public-chain observer	C for proof and commitment contents; empirical for modeled metadata	C	n/a	C: outcome recorded
Untrusted requester	C against disclosure from public artifacts; adaptive-query leakage not evaluated	C: cannot forge a proof or alter its bound authorization context	n/a	C: pseudonymous request context and outcome recorded on-chain
Workflow-following TPS	– against the TPS itself; T against onward disclosure	C for the committed relation; T/E for unsigned evidence provenance	T	C for submitted decisions; – for refusal
Compromised or colluding TPS	–	C for the committed relation and signed claims; E for externally authenticated evidence; – for unauthenticated evidence provenance	–: may refuse to prove	C for submitted decisions; – for refusal
Forged credential	n/a	C: signature verified under the policy-bound issuer key	n/a	n/a
Unauthorized or revoked issuer	n/a	E: issuer authorization and revocation status are external	n/a	C for the on-chain decision record; E for determining issuer status

Note: C: cryptographic; T: trust in the TPS following the prescribed workflow; E: external or lifecycle function; –: not provided by the prototype; n/a: not applicable. In the evaluated provider-operated deployment, the provider and TPS are co-located, so all TPS-dependent confidentiality and liveness assumptions are concentrated at the provider.

Location in the revised manuscript: Section "Discussion," paragraph on the TPS trust trade-off.

The TPS trust assumption is the main architectural tradeoff. Moving confidentiality-sensitive proof generation to the TPS keeps policy-bound proving information away from requesters and public smart contracts while preserving on-chain verifiability, but it shifts part of the confidentiality burden to the service layer; the provider-operated TPS used here concentrates that burden at the provider rather than at an independent service (Section 4). This tradeoff leaves on-chain verifiability intact but does affect liveness and accountability: the TPS or provider can refuse to produce a proof, and such a refusal may be indistinguishable from a legitimate denial at the on-chain interface.

Location in the revised manuscript: Section "Discussion," final paragraph.

> Three directions follow. First, adding an explicit chain/contract domain and an authenticated submission channel would extend context binding beyond a single deployment and reduce front-running risk. Second, accountable proof obligations—for example, on-chain commitments to serve requests together with timeout-with-evidence mechanisms—would make proof refusals attributable. Third, the reduced-trust TPS instantiations of Section 4 (collaborative-SNARK or TEE-backed proving) would relax the trust concentration itself, including the attribute-origin limitation; realizing and evaluating them is left to future work.

Comments 5: *Empirically-grounded claims of privacy must be carefully scoped. The evaluation assumes a passive, metadata-only public-chain adversary, and the effective policy-dependent attack features ultimately reduce primarily to commit and access gas measurements, as several other transcript characteristics are constant by construction. The authors rightly point out that their threat model does not cover timing of transactions, ordering of transactions, mempool/broadcast behavior of TPSs, off-chain communication of the TPS, adaptive probing of requesters, and compromised behavior of TPSs. These exclusions are important. The wording should not be interpreted as establishing general policy confidentiality, and the result should be qualified as empirical resistance against the modeled public-chain metadata attacks.*

Response 5: We agree and have adopted the reviewer's formulation throughout. The abstract, the opening of the Discussion, and the Conclusion now describe the result as empirical resistance against the modeled public-chain metadata attacks, and the Discussion states the exclusions in one place rather than leaving them implicit in the threat model.

The revision also gave us an occasion to apply this scoping to our own results. The evaluation was re-run on the revised prototype, whose proof relation binds the program length, the authorization context, and the issuer-signed profile claims (Comments 1–3), on a new corpus of 1,077 policies and 1,773 Polygon Amoy executions. Under the multi-seed criterion, two of the 45 supervised tasks now meet the failure condition: the ICD-10 chapter task for chapter ch5 and the nesting-depth contrast medium/deep. We report both as failures. Because the gas features are, as the reviewer notes, the only non-constant observables, we could examine what the classifier separates: the access gas is the randomized pad target plus a near-constant 4,661–4,750 gas, the capture log records every target, and subtracting it removes the separability entirely (for ch5, the gas and target AUCs are both 0.601 and the residual AUC is 0.472; rerunning the unchanged protocol on the residual returns both tasks to PASS). The two failures are therefore coincidences of the padding's own random draws with a label—a false-positive mode of the criterion that we quantify in the response to Comment 6—and not information about the policy. We chose to keep the criterion and the failures as reported and to add the diagnostic, rather than to adjust thresholds after the fact.

The gas-padding ablation, re-run on the same corpus, sharpened the scoping in one more respect: the channel that the un-padded transcript exposes is not the policy logic, which the fixed circuit already hides (the un-padded access gas of 1,119 executions spans 276 gas), but the calldata byte pricing of the requester pseudonym, whose encoding width in our harness correlates with whether a policy activates the profile predicate. We state this in the revised ablation section because it illustrates the reviewer's point: on a public chain, any deterministic encoding difference that correlates with the policy is a channel, and the empirical claim covers the channels we modeled.

Changes in the manuscript. The Abstract identifies the modeled adversary, the Discussion states the empirical result and all excluded channels, and the Conclusion repeats the limitations without claiming general policy confidentiality.

Location in the revised manuscript: Abstract.

> In blockchain-based healthcare data sharing, consent policies are expected to be enforced verifiably and auditably, but public-chain transparency may expose sensitive policy information through on-chain representations, verification interfaces, and execution artifacts. Existing privacy-preserving solutions mainly protect data contents, identities, or transactions, not the governing consent policies. We propose the Trusted Proof Service-assisted Zero-Knowledge Policy Interpreter (TPS-ZKPI), which lets an on-chain verifier check authorization decisions while limiting leakage about policy content, identity, and structure. TPS-ZKPI combines fixed-length policy encoding, a reusable zero-knowledge policy interpreter, commitment-based policy binding, and public-chain transcript normalization. We evaluate TPS-ZKPI on Polygon Amoy against metadata-only public-chain adversaries with supervised and unsupervised inference attacks. Under a multi-seed leakage criterion, no attack recovers hidden policy information reproducibly above chance; a paired ablation attributes the gas-channel protection to the gas padding of transcript normalization, at an overhead of about 46% on access and 38% on commit gas.

Location in the revised manuscript: Section "Discussion," opening two paragraphs.

> The evaluation supports a scoped claim: against the modeled public-chain metadata adversary, TPS-ZKPI provides empirical resistance rather than proven policy confidentiality. Of the 45 supervised tasks, 43 stay below the multi-seed leakage criterion, and unsupervised clustering shows no recoverable structure beyond one small, permutation-significant nesting-depth ARI. The two tasks that meet the criterion—one ICD-10 chapter and the `nestingDepth medium/deep` contrast—are reported as failures, and both are carried by the random pad target rather than by the policy: subtracting the logged target from the access gas returns each to PASS, and replacing the gas features by policy-independent draws reproduces such failures in four of ten replicates. This is a property of the criterion on a corpus in which coverage-layer policies are executed once, so the thirty seeds resample the same random draws; we keep the criterion as stated and document the diagnostic, since lowering the significance threshold or raising the AUC threshold to absorb these cases would weaken it for genuine leaks. The gas-padding ablation attributes the gas-channel protection to randomized gas padding: without it the same attacks recover the policy family with AUCs up to 0.90. Content hiding itself rests on the commitment-only interface and the content-independent transcript normalization; padding is the defense-in-depth layer that absorbs whatever deterministic gas differences remain, of which the ablation exhibits one that does not come from the policy logic at all but from the byte encoding of a public transaction argument. Together with fixed-length policy representation, reusable interpretation, and commitment binding, these mechanisms limit the extent to which public-chain observers infer hidden policy content or structural properties from authorization transactions.
>
> The scope of this claim is the adversary of Section 6.1: a passive observer of commitments, proofs, public parameters, calldata, events, and gas, on a corpus of generated policies. Timing and ordering of transactions, mempool and broadcast behaviour, off-chain communication with the TPS, adaptive requester probing, repeated access attempts, correlations between authorization outcomes and policy classes (every request in the corpus is granted), and compromised TPS behaviour are not modeled, and the results should not be read as establishing general policy confidentiality against them.

Location in the revised manuscript: Section "Conclusion," second and third paragraphs.

> The evaluation shows what such a design can and cannot promise. On a live testnet corpus, a supervised attacker with policy-disjoint training data and an unsupervised clustering attacker recover neither policy content, nor policy identity, nor structure and parameters from the normalized transcript beyond what the padding's own random draws produce, and the paired ablation shows that the padding is what removes the last policy-correlated gas signal. The ablation also delivers a cautionary result: the un-padded channel it exposes is not the policy logic, which the fixed circuit already hides, but the byte-level pricing of one public transaction argument—a reminder that on a public chain any deterministic encoding difference that correlates with the policy is a channel, and that the defense must cover the class rather than the instances one anticipated.
>
> The results hold for a metadata-only public-chain adversary and are empirical. Policy confidentiality against the TPS itself is not cryptographic but rests on the service following its workflow, and the evaluation's provider-operated TPS concentrates that trust at the provider. The supported policy class is an access-time checker for bounded Boolean policies over standardized terms: institutional or ethics approval, downstream purpose limitation, post-access duties, delegation, and credential revocation stay outside the proved statement. On-chain policy revocation and evolution govern future authorization decisions but do not retract data already released or enforce downstream cessation or deletion. Timing, ordering, mempool, and off-chain channels, adaptive probing, and correlations between outcomes and policy classes are not modeled, and the leakage criterion has a measurable false-positive rate on corpora that execute each policy once.

Comments 6: *Provide additional justification for the PASS/FAIL leakage criteria. In the manuscript, a failure is defined only if the mean AUC is greater than 0.55 and at least 50% of the 30 seeded permutation tests are significant. This leads to a criterion based on reproducibility, although the exact thresholds seem to be application-defined. The authors need to better justify the choice of 0.55 and 50% and ideally perform a sensitivity analysis to show that the overall conclusions are robust to reasonable alternative thresholds.*

Response 6: We thank the reviewer for this request; the thresholds are indeed application-set, and the revised methodology now justifies each leg and reports a sensitivity analysis.

Justification. The AUC threshold of 0.55 requires a mean separation of more than 0.05 from chance, while the 50% threshold requires the permutation test to reject in at least 15 of the 30 policy-disjoint splits. Because the splits overlap, the significant-seed fraction is a stability measure rather than a combined p-value. Both conditions must hold for a FAIL verdict.

Sensitivity analysis. The analysis pipeline now records every task's per-seed AUCs and permutation p-values, and a new script re-applies the criterion over AUC thresholds $\{0.52, 0.55, 0.58, 0.60\}$ and significance fractions $\{0.3, 0.4, 0.5, 0.6, 0.7\}$ without retraining. Whenever the significance fraction is at least 0.5, at most the two tasks that fail at the of-record thresholds (ICD ch5 and nesting-depth medium/deep) fail; lowering the fraction to 0.3–0.4 adds at most two SPP contrasts; raising it to 0.6 fails nothing. The PI verdict is unchanged in every cell. Thus, the exact failure count varies with the operating point, but the qualitative conclusion—that the evaluation shows no widespread leakage—is stable across the grid.

Null calibration. We also calibrated how often the operational criterion produces spurious flags under policy-independent features. The padded access gas contains a per-policy random

draw (the pad target), and the coverage-layer policies are executed once, so the 30 seeds resample the same draws; a coincidence between the draws and a label reproduces across seeds. Replacing the two gas features by policy-independent draws from the padding bands and rerunning the entire pipeline ten times produced at least one spurious PCH failure in four of the ten replicates (a per-task rate of about 3%; none on SPP). The two failures in the of-record run are of exactly this kind: subtracting the logged target from the access gas returns both to PASS (Comment 5). We kept the criterion as stated, report the failures, and treat a failure as a prompt to examine what the classifier separates; the manuscript now says so.

Location in the revised manuscript: Section "Privacy Evaluation," subsection "Attack methodology," paragraph "Operational thresholds and sensitivity": the operational AUC and stability conditions, threshold-sensitivity and null-calibration results, and the paired-ablation basis for the effectiveness claim.

> The PASS/FAIL labels summarize residual distinguishability under the evaluated attacks; they do not by themselves establish the effectiveness of the defense. The AUC threshold imposes an operational effect-size floor of 0.05 above chance, while the stability condition requires the permutation-test p-value to be below 0.05 in at least 15 of the 30 policy-disjoint splits. Because the splits overlap, the significant-seed fraction is a stability measure rather than a combined p-value.
>
> Because these thresholds are operational choices, we reapply the criterion over AUC thresholds of 0.52–0.60 and significant-seed fractions of 0.3–0.7. At fractions of at least 0.5, at most the two tasks discussed in the PCH and SPP result sections fail; lowering the fraction to 0.3–0.4 adds at most two SPP contrasts, whereas raising it to 0.6 yields none. In ten null reruns with policy-independent gas features, four contain a spurious PCH failure (a 3% per-task frequency), while none produces an SPP failure. We therefore treat FAIL as a diagnostic flag and examine the two observed failures in the corresponding result sections without changing their verdicts.
>
> The effectiveness of gas normalization is assessed separately by the paired gas-normalization ablation. Using the same policies, features, splits, and classifiers, the attacks recover signal from the unpadded transcript, and that signal collapses after padding. Thus, the PASS/FAIL criterion summarizes residual attack performance, whereas the paired reduction provides the primary empirical evidence that padding suppresses the modeled gas channel.

Comments 7: *Can you expand on the real cost of randomizing gas padding? The gas padding ablation is important, as it shows that policy-dependent gas signals are more recoverable when the padding is removed. But it costs about 38% in access-gas and 40% in commit-gas overhead to provide this protection. The manuscript would benefit from a more explicit discussion of this privacy-cost trade-off, e.g. whether the overhead remains practical for realistic transaction volumes and whether the padding strategy remains effective when policies or circuits become more complex.*

Response 7: We expanded the cost discussion in three ways.

Re-measured overhead. The ablation was re-run on the revised system and the new corpus (a 1,119-execution subset, executed without padding three days after the padded of-record run, on a fresh deployment). The overhead is now 46.1% on access gas (723,462 → 1,056,674) and 38.4% on commit gas (219,737 → 304,098). The increase on access relative to the submitted version has two causes that the manuscript now explains: the revised relation adds an on-chain Poseidon recomputation of the context digest, and Polygon Amoy prices the pairing precompiles

higher than the local EVM (the same verifier costs 243,000 gas locally and 372,000 on Amoy), so the padding band had to be positioned higher.

Practicality at volume. The overhead is a fixed premium per transaction—roughly 330,000 gas on access and 85,000 on commit at the band used here—so it scales linearly with transaction volume and does not compound. At the 30 gwei tip of the evaluation a padded access transaction costs about 0.032 POL against 0.022 POL unpadded. The Discussion now states this together with the deployment rule the band follows: its base must lie above the bare access cost measured on the target network, which is calibrated once per network, and its width (±10% here) determines how large a deterministic difference it can submerge.

Effectiveness under more complex policies or circuits. Neither the bare cost nor the band depends on the policy: the fixed circuit gives every proof the same verification work and the same five public signals, so a more complex policy changes nothing on-chain. To show how the prover-side cost scales, we added an offline sweep of the program length ($L = 32, 64, 128$ with N and M fixed; new cost-scaling table): constraints grow from 65,000 to 134,000 and proving time from 0.38 to 0.73 s per proof on a laptop-class prover, while the verifier contract, its gas, and the padding band are unchanged. Richer predicates would enter through the predicate slots and their gadgets, again adding prover work and nothing on-chain, because predicate parameters are bound through the fixed-format parameter commitment rather than exposed as public inputs. The revised ablation section also identifies what the padding must absorb: the un-padded channel is a sub-kilogas calldata-pricing difference in a public argument, which a ±105,000-gas band submerges regardless of its origin; padding cannot hide a difference larger than its band, and we say so.

Changes in the manuscript. The Discussion now explains the per-transaction privacy premium and network calibration rule. The Ablation subsection reports the measured overhead and the new scaling experiment, whose complete table is reproduced below.

Location in the revised manuscript: Section “Discussion,” paragraph “The cost of padding.”

> Within this scope, the residual cost is the randomized padding’s gas overhead: on the ablation subset, about 46% on access gas and 38% on commit gas. Three observations bound its practical weight. First, the overhead is a fixed premium per transaction—roughly 330,000 gas on access and 85,000 on commit at the band used here—so it scales linearly with transaction volume and does not compound: at the 30 gwei tip of the evaluation, a padded access transaction costs about 0.032 POL against 0.022 POL unpadded. Second, neither the bare cost nor the band depends on the policy: the fixed circuit gives every proof the same verification work and the same five public signals, and the prover-side cost of longer or richer policies (Table 16) is paid off-chain. Third, the band must be positioned above the bare cost on the target network, which differs between networks (the same verifier costs 243,000 gas on a local EVM and 372,000 on Amoy, whose pairing precompiles are priced higher); a deployment therefore calibrates the base target once per network, and the width of the band, ±10% here, is what determines how large a deterministic difference it can submerge. Padding cannot, of course, hide a difference larger than its band, and it does not address channels that are not gas.

Location in the revised manuscript: Section “Privacy Evaluation,” subsection “Ablation,” paragraph “Cost.”

> Padding is not free: over the subset it raises mean commit gas from 219,737 (B1) to 304,098 (B2), a 38.4% increase, and mean access gas from 723,462 to 1,056,674,

a 46.1% increase. The padded access cost is set by the padding band, not by the policy: the TPS draws the target uniformly from 945,000–1,155,000 gas, and the base of that band is chosen above the bare access cost measured on the target network (748,000 gas on Amoy, whose pricing of the pairing precompiles is higher than the local EVM's) so that the whole band stays above real usage. At the 30 gwei tip used throughout, one padded access transaction costs about 0.032 POL and one commit about 0.009 POL; the overhead is therefore a fixed per-transaction premium of roughly 330,000 gas on access and 85,000 on commit, independent of transaction volume, and it does not grow with policy complexity, because neither the bare cost nor the band depends on the policy. Table 15 gives the single-point cost characterization of the evaluated prototype configuration at DEPTH = 64. The circuit compiles to 81,740 R1CS constraints; across the 1,773 Polygon Amoy executions, mean witness and proving times on the two-core capture host are 763.6 ms and 2,884.2 ms, respectively. Gas, logs, calldata lengths, and transaction objects are taken from the same executed dataset, whose transactions use the context-bound five-signal statement. Table 16 shows how the prover-side cost scales when the program length grows from DEPTH = 32 to 128 with N and M fixed: constraints, proving-key size, and proving time grow roughly linearly with DEPTH (from 65,000 to 134,000 constraints and from 0.38 to 0.73 s per proof on a laptop-class prover), while the statement keeps its five public signals, so the verifier contract, its gas, and the padding band are unchanged. Richer predicates would enter through M and the per-slot gadgets rather than through DEPTH; each additional slot adds its gadget's constraints to the prover and nothing to the on-chain cost, since predicate parameters are bound through the fixed-format parameter commitment rather than exposed as public inputs.

Location in the revised manuscript: Section "Privacy Evaluation," new Table 16, "Prover-side cost as the policy program length grows."

Table 16. Prover-side cost as the policy program length DEPTH grows, with $N = 13$, $M = 11$ and the public interface fixed. On-chain verification cost does not change: the statement keeps five public signals at every depth.

DEPTH	R1CS constraints	Proving key (MB)	Witness (ms)	Prove (ms)	Verify (ms)
32	64,945	38.7	282	382	11
64	81,740	52.2	343	494	10
128	134,009	87.5	542	730	11